

Informe Técnico

Tema 9: Construcción de rutas de aprendizaje

Proyecto final de IA y Simulación

Daniela de la Caridad Guerrero Álvarez

Grupo C311

Asignatura: Inteligencia Artificial y Simulación

Índice

1. Introducción	4
2. Presentación general del sistema	4
3. Descripción del problema	4
4. Modelado formal	5
5. Dataset utilizado	6
5.1. Origen del dataset	6
5.2. Estructura del grafo de dependencias	6
5.3. Por qué este dataset cumple las condiciones del problema	7
5.4. Instancias de prueba	7
6. Rol del LLM en el sistema	8
6.1. Pipeline de evaluación	8
6.2. Técnica few-shot para calibrar la escala	8
6.3. Manejo de errores y resiliencia	9
6.4. Evaluación por lotes (batch)	9
7. Diseño del algoritmo	9
7.1. Programación dinámica (DP)	9
7.2. Algoritmo voraz (greedy) por densidad de utilidad	10
7.3. Muestreo Monte Carlo	10
7.4. Solver híbrido	10
7.5. Análisis de robustez	10
8. Implementación completa del sistema	11
8.1. Fase 1: Modelado y carga de datos	11
8.2. Fase 2: Evaluación semántica con el LLM	11
8.3. Fase 3: Solver híbrido de optimización	12
9. Configuración del uso del LLM	12
10. Instrucciones de ejecución	12
10.1. Instalación	12
10.2. Configuración de credenciales	13
10.3. Ejecución por fases	13
11. Diseño experimental	13
11.1. Conjunto de instancias utilizadas	14
11.2. Variantes y configuraciones comparadas	14
12. Metodología experimental	14
12.1. Experimento 1: impacto del número de iteraciones Monte Carlo	14
12.2. Experimento 1b: curvas de convergencia	15
12.3. Experimento 2: impacto de la temperatura	15
12.4. Experimento 3: escalabilidad	15
12.5. Experimento 4: análisis de robustez	16
12.6. Procedimiento general y reproducibilidad	16

13.Resultados y análisis	16
13.1. Resultados del Experimento 1: iteraciones Monte Carlo	16
13.2. Resultados del Experimento 1b: curvas de convergencia	17
13.3. Resultados del Experimento 2: temperatura	17
13.4. Resultados del Experimento 3: escalabilidad	18
13.5. Resultados del Experimento 4: robustez	19
14.Limitaciones y posibles mejoras	20
15.Conclusión	21

Resumen

En este informe se presenta el desarrollo de un sistema que ayuda a una persona a armar una ruta de cursos para aprender algo, tomando en cuenta el tiempo que tiene disponible y qué tan relevante es cada curso para lo que quiere lograr. Para esto se combinan dos cosas: un modelo matemático del problema (parecido al clásico problema de la mochila, pero con la diferencia de que algunos cursos dependen de otros) y un modelo de lenguaje grande (LLM) que se encarga de decir qué tan útil es cada curso según el objetivo del usuario. El documento explica paso a paso cómo se modeló el problema, cómo se construyó el dataset de cursos, cómo se integró el LLM, qué algoritmos se usaron para resolver la optimización, y qué resultados se obtuvieron al probar todo con tres instancias de distinto tamaño.

1 Introducción

Cuando una persona quiere aprender algo nuevo, por ejemplo, especializarse en inteligencia artificial o en programación, normalmente se encuentra con un catálogo enorme de cursos y no sabe por dónde empezar. Además, muchos cursos tienen requisitos: no se puede tomar un curso avanzado de redes neuronales sin haber visto antes los fundamentos de álgebra lineal o programación. Sumado a esto, la persona casi siempre tiene un tiempo limitado disponible para estudiar, por lo que no puede simplemente tomar todos los cursos que existen.

Este proyecto nace de esa necesidad: construir un sistema que, dado un catálogo de cursos con sus duraciones y sus prerrequisitos, y dado un objetivo de aprendizaje expresado en lenguaje natural por el usuario (por ejemplo, “quiero especializarme en procesamiento de lenguaje natural”), sea capaz de generar automáticamente una ruta de cursos que cumpla con el tiempo disponible, respete los prerrequisitos, y que además esté lo más alineada posible con lo que la persona realmente quiere aprender.

La parte interesante de este proyecto es que combina dos mundos distintos. Por un lado, está la parte de optimización combinatoria clásica, que es la que se encarga de resolver matemáticamente cuál es el mejor subconjunto de cursos posible dado un presupuesto de tiempo y una red de dependencias. Por otro lado, está el modelo de lenguaje grande (LLM), que es el que entiende el lenguaje natural y puede decir, para cada curso, qué tan relevante es respecto al objetivo declarado por el usuario. Ninguna de las dos partes podría resolver el problema completo por sí sola: el algoritmo de optimización no entiende texto, y el LLM no es bueno resolviendo combinatoria exacta bajo restricciones. Por eso se diseñó un sistema híbrido donde cada componente hace lo que mejor sabe hacer.

2 Presentación general del sistema

El sistema completo se organiza en tres fases que se ejecutan de forma secuencial, y cada fase corresponde a una carpeta de código dentro del proyecto. En la primera fase se define el modelo formal del problema y se construye el dataset de cursos, que forma un grafo dirigido acíclico (DAG) de dependencias. En la segunda fase se integra el LLM, que recorre el catálogo de cursos y le asigna a cada uno una puntuación de utilidad según el objetivo del usuario. En la tercera fase se ejecuta el algoritmo de optimización, que toma las utilidades calculadas por el LLM y busca la mejor combinación de cursos posible dentro del tiempo disponible.

A lo largo del informe se explica cada una de estas fases con el nivel de detalle suficiente para que cualquier persona con conocimientos básicos de programación e inteligencia artificial pueda entender qué se hizo, por qué se hizo de esa manera, y cómo se puede ejecutar el sistema completo desde cero.

3 Descripción del problema

El problema que se quiere resolver se puede resumir de la siguiente manera: se tiene un catálogo de cursos, cada uno con una duración en horas y posiblemente con uno o varios cursos prerrequisito que deben completarse antes. Se tiene también un tiempo máximo disponible, T_{max} , que la persona puede dedicar a estudiar. El objetivo es elegir un subconjunto de cursos tal que:

- La suma de las duraciones de los cursos elegidos no supere T_{max} .
- Si se elige un curso, todos sus prerrequisitos también deben estar en la selección (a esto se le llama **clausura de prerrequisitos**).

- La suma de las “utilidades” de los cursos elegidos sea lo más alta posible, donde la utilidad de cada curso representa qué tan relevante es ese curso para el objetivo de aprendizaje del usuario.

La parte difícil de este problema es que la utilidad de un curso no es un número que venga dado de antemano en el dataset. No existe una columna “utilidad” en los datos originales, porque la utilidad depende de qué quiere aprender la persona, y eso solo se sabe en el momento en que el usuario hace su consulta. Por eso es necesario un componente que, en tiempo de ejecución, lea la descripción de cada curso y el objetivo del usuario, y produzca un número que represente esa relevancia. Ese componente es el LLM.

Una vez que cada curso tiene asignado un número de utilidad entre 1 y 10, el problema se convierte en una versión del clásico **problema de la mochila (knapsack)**, pero con una complicación adicional: las dependencias entre cursos. A este tipo de problema se le conoce en la literatura como *knapsack with precedence constraints*, y se sabe que es un problema NP-difícil en general, es decir, que no existe (hasta donde se sabe) un algoritmo que lo resuelva de forma exacta y rápida para cualquier tamaño de entrada. Esto justifica por qué, además de un algoritmo exacto, también se implementaron heurísticas como un algoritmo voraz (greedy) y un muestreo Monte Carlo.

4 Modelado formal

Para poder programar el problema, primero hubo que escribirlo en términos matemáticos precisos. El dominio de conocimiento se representa como un grafo dirigido acíclico $G = (V, E)$, donde V es el conjunto de cursos (los nodos del grafo) y E es el conjunto de relaciones de prerequisite (las aristas dirigidas). Si existe una arista (v_i, v_j) , eso quiere decir que el curso v_i es prerequisite del curso v_j , es decir, que v_j no se puede tomar sin haber tomado antes v_i .

Que el grafo sea acíclico significa que no puede pasar que el curso A dependa del curso B, y al mismo tiempo el curso B dependa (directa o indirectamente) del curso A, porque eso crearía una dependencia circular imposible de satisfacer. Esta propiedad se puede comprobar de forma automática usando un recorrido topológico, por ejemplo con el algoritmo de Kahn, que va eliminando nodos sin dependencias pendientes hasta procesar todo el grafo; si al final quedan nodos sin procesar, es porque hay un ciclo.

Cada curso v tiene dos atributos numéricos importantes. El primero es la duración $d(v)$, que es un número positivo medido en horas, y en el dataset de este proyecto está en el rango $[10, 60]$ horas. El segundo es la utilidad $u(v)$, que es un número en el rango $[1, 10]$ y que, como se explicó antes, no se conoce de antemano: es calculado dinámicamente por el LLM a partir de la descripción del curso y del objetivo declarado por el usuario. Formalmente esto se escribe como:

$$u(v) = \mathcal{F}_{\text{LLM}}(\text{desc}(v), \text{objetivo del usuario}) \in [1, 10]$$

Una ruta de aprendizaje válida es un subconjunto de nodos $S \subseteq V$ que cumple la clausura de prerequisites, es decir:

$$\forall v_j \in S, \forall (v_i, v_j) \in E \Rightarrow v_i \in S$$

En palabras simples: si un curso está en la ruta, todos los cursos que necesita como requisito también deben estar en la ruta. Además, la ruta debe cumplir la restricción dura de tiempo:

$$\sum_{v \in S} d(v) \leq T_{\max}$$

Esta restricción es “dura” porque ninguna solución que la incumpla se considera válida, sin importar qué tan buena sea su utilidad total.

Finalmente, el objetivo es encontrar el subconjunto S^* que maximiza la utilidad acumulada:

$$S^* = \arg \max_{S \subseteq V} \sum_{v \in S} u(v) \quad \text{sujeto a} \quad \sum_{v \in S} d(v) \leq T_{max} \quad \text{y a la clausura de prerrequisitos.}$$

Para poder programar esto, se reescribe usando variables binarias $x_v \in \{0, 1\}$, donde $x_v = 1$ si el curso v se incluye en la ruta y $x_v = 0$ si no. La formulación completa queda así:

$$\begin{aligned} & \max_{x \in \{0,1\}^{|V|}} \sum_{v \in V} u(v) \cdot x_v \\ \text{sujeto a:} \quad & \sum_{v \in V} d(v) \cdot x_v \leq T_{max} \\ & x_{v_j} \leq x_{v_i} \quad \forall (v_i, v_j) \in E \\ & x_v \in \{0, 1\} \quad \forall v \in V \end{aligned}$$

La segunda restricción ($x_{v_j} \leq x_{v_i}$) es la forma matemática de decir la clausura de prerrequisitos: si $x_{v_j} = 1$ (el curso v_j está seleccionado), entonces x_{v_i} también debe ser 1, porque si x_{v_i} fuera 0 se violaría la desigualdad $1 \leq 0$.

Como ya se mencionó, esta formulación es una generalización del problema de la mochila con restricciones de precedencia, y es NP-difícil. Por eso el sistema combina un algoritmo exacto (que solo es práctico para instancias pequeñas) con heurísticas (que son rápidas pero no garantizan el óptimo) y con el LLM, que actúa como un “oráculo” que aporta la señal de utilidad necesaria para guiar tanto al algoritmo exacto como a las heurísticas.

5 Dataset utilizado

5.1 Origen del dataset

El dataset usado en este proyecto no se obtuvo de una fuente externa, sino que fue **generado de forma sintética** para este trabajo. Se decidió generarlo en lugar de usar uno existente porque era necesario tener control total sobre la estructura de dependencias (que formara un DAG perfectamente verificable), sobre las duraciones de los cursos, y sobre el contenido textual de las descripciones, de manera que el LLM tuviera información suficiente para hacer un análisis semántico con sentido.

El dataset, llamado `cursos.json`, contiene **35 cursos sintéticos** del dominio de Ciencia de Datos e Inteligencia Artificial. Cada curso tiene los siguientes campos: un identificador único (`id`), un título legible (`título`), una descripción de tres a cinco líneas escrita en lenguaje natural con temas abstractos y prácticos (`descripcion`), una duración en horas entre 10 y 60 (`duracion_horas`), y una lista de identificadores de los cursos que son prerrequisito (`prerrequisitos`).

5.2 Estructura del grafo de dependencias

Los 35 cursos se organizan en **7 niveles de profundidad**, numerados de L0 a L6, donde cada nivel tiene 5 cursos. Los cursos del nivel L0 (identificados como CS_101 a CS_105) no tienen ningún prerrequisito, es decir, son los nodos raíz del grafo. Los cursos de cada nivel siguiente solo pueden tener como prerrequisitos cursos de niveles anteriores, nunca de niveles posteriores. Esto garantiza, por construcción, que el grafo sea un DAG: el orden topológico simplemente sigue la secuencia $L0 \rightarrow L1 \rightarrow L2 \rightarrow L3 \rightarrow L4 \rightarrow L5 \rightarrow L6$, y nunca existe una arista que vaya de un nivel más alto a uno más bajo.

La siguiente tabla resume la organización del catálogo completo:

Nivel	IDs	Cantidad de cursos	Prerrequisitos
L0	CS_101 – CS_105	5	Ninguno (nodos raíz)
L1	CS_201 – CS_205	5	Cursos de L0
L2	CS_301 – CS_305	5	Cursos de L1
L3	CS_401 – CS_405	5	Cursos de L2
L4	CS_501 – CS_505	5	Cursos de L3
L5	CS_601 – CS_605	5	Cursos de L4
L6	CS_701 – CS_705	5	Cursos de L5

La duración total de todos los cursos del catálogo suma 1295 horas, lo cual da una idea de la escala del problema cuando se trabaja con el dataset completo.

5.3 Por qué este dataset cumple las condiciones del problema

Para que el dataset sea útil para el problema definido, debía cumplir varias condiciones, y el proceso de generación se diseñó específicamente para garantizarlas:

1. **Debía formar un DAG verificable.** Como se explicó, la estructura por niveles asegura que nunca hay ciclos, porque las dependencias solo apuntan “hacia atrás” en la secuencia de niveles. Esto se puede comprobar automáticamente con el algoritmo de Kahn implementado en el código (método `validate_dag`).
2. **Debía tener duraciones variadas pero acotadas.** Se eligió un rango de 10 a 60 horas por curso, lo suficientemente amplio para que existan decisiones de compromiso (cursos cortos versus cursos largos) pero no tan amplio como para que un solo curso domine completamente el presupuesto de tiempo.
3. **Debía tener descripciones ricas en lenguaje natural.** Cada curso incluye una descripción de tres a cinco líneas que menciona temas concretos (por ejemplo, arquitecturas de redes neuronales, técnicas específicas, herramientas) para que el LLM tenga información suficiente para juzgar la relevancia semántica respecto a distintos objetivos de usuario.
4. **Debía permitir presupuestos de tiempo que generen decisiones reales.** Es decir, que T_{max} no sea tan grande que se puedan tomar todos los cursos, ni tan pequeño que no se pueda tomar casi nada.

5.4 Instancias de prueba

A partir del catálogo completo se generaron tres instancias de prueba de tamaño creciente, pensadas para estresar el sistema de forma gradual:

Instancia A (pequeña): contiene 10 cursos y $T_{max} = 80$ horas. Tiene una estructura mayormente lineal (una cadena de cursos donde cada uno depende del anterior, con una rama paralela). Esta instancia es tan pequeña que su solución óptima se puede verificar a mano, y sirve como prueba de sanidad (*sanity check*) para confirmar que el algoritmo funciona correctamente antes de pasar a instancias más grandes.

Instancia B (mediana): contiene 22 cursos y $T_{max} = 200$ horas. Tiene ramificaciones más interesantes: por ejemplo, hay cursos que requieren tres prerrequisitos simultáneos, y hay nodos donde convergen dos ramas distintas del grafo. Esta instancia sirve para observar cómo el algoritmo maneja rutas competidoras que llevan al mismo destino con distinto costo y distinta utilidad.

Instancia C (grande): contiene los 35 cursos completos y $T_{max} = 300$ horas, lo que representa apenas un 23 % del tiempo total del catálogo (1295 horas). Esta es la instancia más exigente: el espacio de posibles combinaciones es enorme, y el presupuesto de tiempo obliga

a tomar decisiones de alta discriminación semántica, es decir, el LLM se vuelve crítico para distinguir qué cursos realmente vale la pena incluir.

La tabla siguiente compara las tres instancias:

Parámetro	Instancia A	Instancia B	Instancia C
Número de cursos $ V $	10	22	35
Número de dependencias $ E $	~ 8	~ 22	~ 42
T_{max} (horas)	80	200	300
Profundidad máxima del DAG	6	4	7
Tipo de ramificación	Lineal	Moderada	Alta / cruzada

6 Rol del LLM en el sistema

Es importante dejar claro qué hace y qué no hace el LLM en este sistema, porque a veces se podría pensar que el modelo de lenguaje es el que “decide la ruta”, y eso no es correcto. El LLM **no toma decisiones finales** ni resuelve la parte de optimización. Su trabajo está muy acotado: actúa como un **oráculo de utilidad semántica**.

Esto significa que, para cada curso del catálogo, el LLM lee su descripción en lenguaje natural y la compara con el objetivo declarado por el usuario, y como resultado produce un número entre 1 y 10 que representa qué tan relevante es ese curso. Esa es toda su responsabilidad. Una vez que cada curso tiene su número de utilidad, el LLM ya no participa más: el resto del trabajo (decidir qué combinación de cursos es la mejor dado el presupuesto de tiempo y las dependencias) lo hace el algoritmo clásico de optimización, que solo trabaja con números.

Esta separación de responsabilidades es deliberada y es uno de los puntos más importantes del diseño. El LLM entiende lenguaje pero no es bueno garantizando que una solución respete restricciones combinatorias exactas; el algoritmo de optimización es exacto con los números pero no entiende lenguaje. Al separar ambas tareas, cada componente hace lo que mejor sabe hacer, y el resultado es un sistema más confiable que si se le pidiera al LLM que “decidiera la ruta completa” directamente.

6.1 Pipeline de evaluación

El proceso de evaluación de cada curso sigue estos pasos: primero se construye un *prompt* para el LLM, que tiene dos partes. La primera parte es el **system prompt**, que es siempre igual sin importar el curso que se esté evaluando: contiene el rol que debe asumir el modelo (“eres un experto en diseño curricular”), una tabla con los criterios de puntuación, tres ejemplos ya resueltos (técnica conocida como *few-shot*), y el formato exacto en que debe responder. La segunda parte es el **user prompt**, que sí cambia en cada llamada: contiene el objetivo del usuario y la descripción del curso que se está evaluando en ese momento.

Una vez armado el prompt, se llama a la API del modelo de lenguaje pidiendo que la respuesta venga en formato JSON (lo que se llama “JSON mode”), y con una temperatura baja (0.1) para que las respuestas sean lo más consistentes posible entre ejecuciones. La respuesta del modelo se valida con Pydantic, una librería que verifica que los datos tengan el tipo y el rango correctos (por ejemplo, que la utilidad sea un número entero entre 1 y 10, y que la justificación tenga entre 20 y 600 caracteres). Si la respuesta del LLM no pasa esta validación, o si hay un error de red, el sistema asigna automáticamente una puntuación neutra de 5/10 al curso y continúa con el siguiente, para que un fallo puntual no detenga todo el proceso.

6.2 Técnica few-shot para calibrar la escala

Una de las decisiones de diseño más importantes fue incluir tres ejemplos resueltos dentro del system prompt, cada uno representando una zona distinta de la escala de utilidad: uno

con utilidad alta (9), uno con utilidad media (5) y uno con utilidad baja (2), todos usando el mismo objetivo de usuario (especialización en NLP y LLMs) pero aplicados a cursos de distintos dominios. La idea detrás de esto es que, sin ejemplos, un modelo de lenguaje tiende a “colapsar” sus respuestas hacia los extremos (poner siempre 1 o 10) o hacia el centro (poner siempre 5). Mostrarle ejemplos reales de cómo varía la puntuación según la distancia entre el dominio del curso y el objetivo del usuario ayuda a que el modelo calibre mejor su propia escala.

6.3 Manejo de errores y resiliencia

Como el LLM es un componente externo (depende de una API de red), el sistema implementa dos niveles de protección. El primer nivel son los reintentos automáticos: si hay un error de conexión o se agota la cuota de la API, el sistema reintenta hasta cuatro veces, esperando cada vez un poco más de tiempo (2, 4, 8 y 16 segundos), usando la librería `tenacity`. El segundo nivel es el fallback de puntuación neutra: si después de los reintentos sigue fallando, o si la respuesta no tiene el formato correcto, el curso recibe automáticamente una utilidad de 5/10 y se marca como “evaluación no exitosa”, de manera que el algoritmo de optimización pueda seguir funcionando sin interrupciones y, si se quiere, se pueda hacer un análisis posterior de cuántos cursos quedaron sin evaluar correctamente.

Además, el cliente LLM implementado (`client.py`) soporta múltiples proveedores (Gemini, Groq y OpenAI), de manera que si el proveedor principal falla por falta de cuota o disponibilidad, el sistema puede cambiar automáticamente a un proveedor de respaldo configurado en las variables de entorno.

6.4 Evaluación por lotes (batch)

Para instancias grandes como la Instancia C (35 cursos), evaluar un curso a la vez resulta muy costoso en términos de tokens consumidos, porque el system prompt (que es bastante largo, con los ejemplos few-shot incluidos) se repite en cada llamada. Por eso se implementó una versión alternativa (`batch_evaluator.py`) que agrupa varios cursos en una sola llamada al LLM, reduciendo el consumo de tokens en aproximadamente un 90 % comparado con el modo individual. Esta versión es la recomendada cuando se trabaja con el catálogo completo o cuando la cuota de la API es limitada.

7 Diseño del algoritmo

Una vez que cada curso tiene asignada su utilidad por el LLM, entra en acción el algoritmo de optimización. En lugar de implementar un único método, se implementaron tres estrategias distintas y un cuarto componente que las combina y elige la mejor.

7.1 Programación dinámica (DP)

La primera estrategia es una solución exacta basada en programación dinámica. Para instancias pequeñas (hasta 20 cursos) se usa una versión por **máscara de bits**: cada subconjunto posible de cursos se representa como un número binario de n bits, donde cada bit indica si ese curso está incluido o no. El algoritmo recorre todas las combinaciones posibles (hasta 2^n) y va guardando, para cada combinación factible (que respeta prerequisites y presupuesto de tiempo), la de mayor utilidad. Esta versión garantiza encontrar la solución óptima, pero su costo crece exponencialmente con el número de cursos, por lo que solo es práctica para instancias pequeñas como la Instancia A y, en el límite, la Instancia B.

Para instancias más grandes (más de 20 cursos) se usa una versión heurística de programación dinámica, similar a la solución clásica del problema de la mochila con una tabla de dos dimensiones (curso, presupuesto restante). Esta versión es mucho más rápida, pero tiene una

limitación importante: la reconstrucción de la solución puede violar la clausura de prerequisites, por lo que después de reconstruir la selección se aplica un paso adicional que elimina los cursos cuyos prerequisites no quedaron incluidos. Esto hace que esta versión sea rápida pero **no garantice el óptimo**.

7.2 Algoritmo voraz (greedy) por densidad de utilidad

La segunda estrategia es un algoritmo voraz simple y rápido. Se calcula, para cada curso, su “densidad” de utilidad, definida como $u(v)/d(v)$, es decir, cuánta utilidad aporta el curso por cada hora invertida. Luego se ordenan todos los cursos de mayor a menor densidad, y se va recorriendo esa lista intentando agregar cada curso a la ruta, siempre verificando que sus prerequisites ya estén incluidos y que quede suficiente presupuesto de tiempo. Este proceso se repite en varias pasadas hasta que ya no se pueda agregar ningún curso más. Es un método determinista (siempre da el mismo resultado para los mismos datos) y muy rápido, aunque tampoco garantiza la solución óptima.

7.3 Muestreo Monte Carlo

La tercera estrategia es un muestreo Monte Carlo guiado por las utilidades. La idea es simular muchas “rutas aleatorias” de la siguiente manera: se empieza con la ruta vacía y un presupuesto igual a T_{max} ; en cada paso se identifican los cursos disponibles (aquellos cuyos prerequisites ya están en la ruta y cuya duración cabe en el presupuesto restante), y se elige uno de ellos al azar, pero no con probabilidad uniforme, sino con una probabilidad proporcional a $\exp(u(v)/T)$, donde T es un parámetro llamado “temperatura”. Esto significa que los cursos con mayor utilidad tienen más probabilidad de ser elegidos, pero no es una elección determinista: siempre hay algo de azar. Este proceso se repite muchas veces (por ejemplo, 2000 o 5000 iteraciones), y al final se devuelve la mejor ruta encontrada entre todas las simulaciones.

El parámetro de temperatura controla el balance entre exploración y explotación: con temperatura muy baja, el algoritmo casi siempre elige el curso de mayor utilidad disponible (comportamiento parecido al greedy), mientras que con temperatura alta, las probabilidades se vuelven más parejas entre todos los cursos disponibles, lo que genera más diversidad de rutas exploradas. Este método es especialmente útil para la Instancia C, donde la programación dinámica exacta sería demasiado costosa.

7.4 Solver híbrido

El cuarto componente es el orquestador, llamado `llm_assisted_solver`. Este componente recibe el problema (con o sin utilidades asignadas), y si hace falta, primero llama al evaluador LLM para completar las utilidades. Luego decide si vale la pena ejecutar la programación dinámica exacta, basándose en el tamaño del espacio de estados ($n \times W$, donde W es el presupuesto discretizado): si ese número es manejable, ejecuta la DP exacta; si no, la omite. En cualquier caso, siempre ejecuta el algoritmo voraz y el muestreo Monte Carlo, porque ambos son rápidos. Al final, compara la utilidad total obtenida por cada uno de los métodos que se ejecutaron y devuelve la mejor ruta encontrada, junto con un resumen comparativo de cómo le fue a cada estrategia.

7.5 Análisis de robustez

Como complemento, se implementó un módulo de análisis de robustez (`robustness.py`). La idea es que las duraciones de los cursos en el dataset son estimaciones nominales: en la realidad, una persona puede tardar más o menos tiempo del estimado en completar un curso. Para modelar esto, se simula que la duración real de cada curso sigue una distribución Gamma centrada en la duración nominal, con un coeficiente de variación `cv` que representa cuánta variabilidad típica se

espera (por ejemplo, $cv=0.20$ representa una variación de aproximadamente $\pm 20\%$). Mediante miles de simulaciones, se estima la probabilidad de que la ruta seleccionada siga siendo factible (es decir, que la suma de las duraciones reales no supere T_{max}), junto con un intervalo de confianza para esa probabilidad. Esto le permite al sistema decirle al usuario algo como: “tu ruta tiene un 87 % de probabilidad de completarse dentro del tiempo presupuestado, considerando que las duraciones reales pueden variar”.

8 Implementación completa del sistema

El proyecto está organizado en módulos de Python, cada uno con una responsabilidad clara, agrupados según la fase a la que pertenecen.

8.1 Fase 1: Modelado y carga de datos

- `problem.py`: define las clases `Course` (un curso, con sus atributos de duración, utilidad y prerequisites) y `LearningPathProblem` (el problema completo, con métodos para validar el DAG, calcular el orden topológico, verificar si una selección es válida y calcular su utilidad y duración total).
- `instance.py`: se encarga de cargar el dataset base (`cursos.json`) y las instancias de prueba desde archivos JSON, ya sea en formato autónomo (con los cursos embebidos) o por referencia (solo una lista de IDs que se buscan en el dataset base). También permite guardar instancias procesadas.
- `run_example.py`: script de demostración que carga el dataset completo y las tres instancias, valida que el DAG sea correcto, y muestra una selección de ejemplo con su utilidad y duración.

8.2 Fase 2: Evaluación semántica con el LLM

- `client.py`: cliente que se comunica con la API del LLM, soporta varios proveedores y maneja reintentos ante errores.
- `prompts.py`: construye el system prompt (con los criterios de evaluación y los ejemplos few-shot) y el user prompt (con el objetivo del usuario y el curso a evaluar).
- `models.py`: define el modelo Pydantic `EvaluacionCurso`, que valida que la respuesta del LLM tenga el formato correcto.
- `evaluator.py`: orquesta la evaluación de todos los cursos de un problema, actualiza los objetos `Course` con las utilidades calculadas, y guarda el resultado en un archivo JSON listo para la Fase 3.
- `batch_evaluator.py`: versión optimizada que evalúa varios cursos en una sola llamada al LLM para ahorrar tokens.
- `cache.py`: implementa una caché local en disco para no repetir llamadas idénticas al LLM.
- `run_fase2.py` y `run_fase2_batch.py`: scripts de demostración de la evaluación, en modo individual y en modo por lotes respectivamente.

8.3 Fase 3: Solver híbrido de optimización

- `baseline.py`: implementa la programación dinámica exacta por máscara de bits, la versión heurística para instancias grandes, y el algoritmo voraz por densidad de utilidad.
- `mc_sampler.py`: implementa el muestreo Monte Carlo guiado por utilidades, además de una función de análisis de convergencia que registra la mejor utilidad encontrada en distintos puntos de la simulación.
- `llm_assisted.py`: el solver híbrido que orquesta la evaluación LLM (si es necesaria) y las tres estrategias de optimización, eligiendo la mejor solución encontrada.
- `robustness.py`: implementa el análisis de robustez bajo incertidumbre en las duraciones.
- `run_fase3.py`: script que carga las tres instancias ya evaluadas, ejecuta el solver híbrido sobre cada una (forzando Monte Carlo en la Instancia C por su tamaño), y guarda los resultados y una comparativa final.

9 Configuración del uso del LLM

La configuración del LLM se maneja mediante variables de entorno, definidas en un archivo `.env` que se crea a partir de una plantilla `.env.example`. Las variables más importantes son las siguientes: `LLM_PROVIDER`, que indica cuál proveedor usar como principal (puede ser `gemini`, `groq` u `openai`); `LLM_FALLBACK_PROVIDER`, que indica un proveedor de respaldo en caso de que el principal falle; las claves de API correspondientes (`GEMINI_API_KEY` u `OPENAI_API_KEY`); `OPENAI_BASE_URL`, necesaria cuando se usa Groq, ya que este proveedor expone una API compatible con la de OpenAI pero en una URL distinta; y opcionalmente `LLM_MODEL` y `LLM_FALLBACK_MODEL`, para especificar qué modelo concreto usar en cada proveedor.

En cuanto a los parámetros de la llamada al modelo, se usa una temperatura baja (0.1) para que las respuestas sean lo más reproducibles posible, y se activa el modo de respuesta JSON, que obliga a que la salida del modelo sea un objeto JSON sintácticamente válido, evitando el problema común de que el modelo agregue texto introductorio antes del JSON (como “¡Claro! Aquí está la evaluación:”).

Las dependencias necesarias para esta parte del sistema se instalan con el siguiente comando:

```
pip install openai pydantic python-dotenv tenacity
```

10 Instrucciones de ejecución

A continuación se describen los pasos para ejecutar el sistema completo desde cero, asumiendo que se tiene el código fuente y Python instalado.

10.1 Instalación

```
git clone <URL_DEL_REPO>
cd Rutas_De_Aprendizaje_Proyecto_IA_Simulacion
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

10.2 Configuración de credenciales

```
copy .env.example .env
```

Después de copiar el archivo, se debe editar `.env` para agregar las claves de API correspondientes al proveedor de LLM que se vaya a usar.

10.3 Ejecución por fases

La Fase 1 valida que el dataset y las instancias sean correctas (que el grafo sea un DAG, que las duraciones y prerequisites estén bien definidos):

```
python src/run_example.py
```

La Fase 2 evalúa el catálogo de cursos con el LLM según un objetivo de usuario de ejemplo, y guarda el resultado enriquecido:

```
python src/run_fase2.py
```

Para instancias grandes, conviene usar la versión por lotes, que consume muchos menos tokens:

```
python src/run_fase2_batch.py
```

La Fase 3 ejecuta el solver híbrido sobre las instancias ya evaluadas y genera los archivos de resultados:

```
python src/run_fase3.py
```

También se puede ejecutar el pipeline completo de principio a fin, indicando la instancia y el objetivo del usuario:

```
python pipeline.py --instancia data/instances/instancia_C_grande.json --objetivo "
    Quiero especializarme en NLP y LLMs"
```

Finalmente, las pruebas automáticas del proyecto se ejecutan con:

```
python -m pytest tests/ -v
```

y los experimentos completos (descritos en la siguiente sección) se ejecutan con:

```
python experiments/run_experiments.py
```

11 Diseño experimental

Una vez que el sistema híbrido quedó implementado y funcionando sobre las tres instancias evaluadas por el LLM (A, B y C), surgió la pregunta natural: ¿qué tan bien funciona realmente? Responder esto a partir de una sola ejecución no es suficiente, porque dos de los tres solvers (Monte Carlo) tienen un componente aleatorio, y porque hay parámetros (número de iteraciones, temperatura) cuyo efecto no es obvio sin medirlo. Por eso se diseñó un conjunto de cinco experimentos, cada uno pensado para responder una pregunta concreta sobre el comportamiento del sistema.

11.1 Conjunto de instancias utilizadas

Todos los experimentos se ejecutan sobre las mismas tres instancias generadas en la Fase 1 y ya evaluadas por el LLM en la Fase 2 (`instancia_A_evaluada.json`, `instancia_B_evaluada.json` e `instancia_C_evaluada.json`). Reutilizar estas instancias evaluadas, en lugar de volver a llamar al LLM en cada experimento, tiene dos ventajas: por un lado se ahorra tiempo y consumo de tokens de la API, y por otro lado se garantiza que todos los experimentos trabajan exactamente sobre los mismos valores de utilidad $u(v)$, de manera que las diferencias observadas entre experimentos se deben únicamente a los parámetros del algoritmo de optimización y no a variaciones del LLM.

Como recordatorio, las tres instancias cubren un rango de tamaños creciente: la Instancia A tiene 10 cursos y $T_{max} = 80$ horas (11 aristas de prerequisite), la Instancia B tiene 22 cursos y $T_{max} = 200$ horas (30 aristas), y la Instancia C tiene 35 cursos y $T_{max} = 300$ horas (53 aristas). Esta progresión permite observar si el comportamiento del sistema cambia de forma cualitativa cuando el problema crece, o si se mantiene estable.

11.2 Variantes y configuraciones comparadas

El diseño experimental compara distintas configuraciones del solver en varios ejes:

- **Número de iteraciones del muestreo Monte Carlo** ($n_{iter} \in \{50, 100, 250, 500, 1000, 2000, 5000\}$), para saber cuántas simulaciones son realmente necesarias.
- **Temperatura del muestreo softmax** ($T \in \{0, 1, 0, 3, 0, 5, 1, 0, 1, 5, 2, 0, 3, 0, 5, 0\}$), para entender el efecto de este parámetro sobre la calidad y la diversidad de las soluciones.
- **Tipo de solver** (programación dinámica exacta, voraz por densidad, y Monte Carlo), comparados en las tres instancias para medir escalabilidad.
- **Tipo de ruta** (óptima del DP, voraz, Monte Carlo, y versiones “parciales” de la ruta óptima que usan solo el 50 %, 70 % o 90 % del presupuesto), comparadas bajo distintos niveles de incertidumbre en las duraciones.

En todos los casos donde interviene el muestreo Monte Carlo, se ejecutan varias réplicas (entre 10 y 15 según el experimento) con semillas aleatorias distintas, partiendo de una semilla base fija (`seed=42`) a la que se le suma un desplazamiento distinto en cada réplica. Esto permite calcular, además del valor medio de cada métrica, su desviación estándar, lo que es fundamental para distinguir un resultado consistente de uno que varió por azar.

12 Metodología experimental

A continuación se describe, para cada experimento, qué pregunta intenta responder, cómo está diseñado exactamente (parámetros, número de réplicas, instancia usada) y dónde se guardan sus resultados. Todos los scripts de experimentos están en la carpeta `experiments/` y generan un archivo `.json` y un archivo `.csv` dentro de `results/`, además de imprimir una tabla resumen en la consola.

12.1 Experimento 1: impacto del número de iteraciones Monte Carlo

Pregunta: ¿cuántas iteraciones del muestreo Monte Carlo son necesarias para obtener una buena solución, y a partir de qué punto agregar más iteraciones deja de valer la pena?

Diseño: se usa la Instancia B (22 cursos, $T_{max} = 200$ h) porque es de tamaño intermedio: ni tan pequeña como para que cualquier configuración encuentre el óptimo trivialmente, ni tan grande como para que el experimento tarde demasiado. Se fija la temperatura en

$T = 1,0$ para aislar el efecto del número de iteraciones. Se prueban los valores $n_iter \in \{50, 100, 250, 500, 1000, 2000, 5000\}$, y para cada valor se ejecutan 10 réplicas con semillas distintas ($seed = 42 + rep \times 100$). Para cada réplica se registra la utilidad obtenida, el tiempo de ejecución, y la iteración en la que se encontró la mejor solución (“iteración de convergencia”). Con las 10 réplicas se calcula la media y la desviación estándar de la utilidad y del tiempo.

12.2 Experimento 1b: curvas de convergencia

Pregunta: dentro de una sola ejecución larga, ¿en qué punto exacto deja de mejorar la mejor solución encontrada hasta el momento?

Diseño: este experimento complementa al anterior. En lugar de comparar ejecuciones completas con distinto número de iteraciones, se ejecuta una sola simulación de 5000 iteraciones y se va registrando, en una serie de puntos de control (*checkpoints*: 10, 25, 50, 100, 200, 350, 500, 750, 1000, 1500, 2000, 3000, 4000 y 5000), cuál es la mejor utilidad encontrada hasta ese punto. Esto se repite para las tres instancias (A, B y C), con 3 réplicas cada una (semillas 42+17, 42+34, 42+51), y se promedian las curvas de las 3 réplicas para obtener una curva media, además de registrar el mínimo y el máximo observado en cada checkpoint.

12.3 Experimento 2: impacto de la temperatura

Pregunta: ¿cómo afecta el parámetro de temperatura del muestreo softmax a la calidad de la solución y a la diversidad de las rutas generadas?

Diseño: de nuevo se usa la Instancia B, esta vez fijando $n_iter = 1000$ (un valor que, según el Experimento 1, ya es suficiente para que el algoritmo converja). Se prueban ocho valores de temperatura: $T \in \{0,1, 0,3, 0,5, 1,0, 1,5, 2,0, 3,0, 5,0\}$, cubriendo desde un comportamiento casi determinista (temperatura muy baja) hasta uno casi uniforme (temperatura alta). Para cada temperatura se ejecutan 15 réplicas (más que en el Experimento 1, porque aquí interesa medir diversidad y se necesita una muestra más grande de soluciones distintas para que esa medición sea confiable). Para cada réplica se guarda la utilidad obtenida, el conjunto de cursos seleccionados y su cardinalidad (número de cursos). Con el conjunto de 15 soluciones por temperatura se calcula el **índice de Jaccard medio**, que es el promedio de la similitud de Jaccard entre todos los pares de soluciones: la similitud de Jaccard entre dos conjuntos A y B se define como $|A \cap B| / |A \cup B|$, y vale 1 si los dos conjuntos son idénticos y 0 si no comparten ningún elemento. A partir de ese promedio se define la **diversidad** como $1 - \text{jaccard_medio}$: una diversidad de 0 significa que todas las réplicas dieron exactamente la misma ruta, y una diversidad cercana a 1 significaría que las rutas son muy distintas entre sí.

12.4 Experimento 3: escalabilidad

Pregunta: ¿cómo se comparan los tres solvers (DP exacto, voraz, Monte Carlo) en términos de calidad de solución y tiempo de cómputo cuando el tamaño del problema crece?

Diseño: se ejecutan los tres solvers sobre las tres instancias (A, B y C). El DP exacto se ejecuta una sola vez por instancia (es determinista), y se comprueba primero que el espacio de estados $n \times W$ (donde W es T_{max} en horas) no supere el umbral de 500.000 estados definido en `llm_assisted.py`; en las tres instancias este umbral no se supera, por lo que el DP exacto se ejecuta en los tres casos. El algoritmo voraz también se ejecuta una sola vez (también es determinista). El Monte Carlo se ejecuta con $n_iter = 2000$ y $T = 1,0$, repitiendo 10 réplicas con semillas $seed = 42 + rep \times 97$, y se reporta la media y la desviación estándar tanto de la utilidad como del tiempo. Para el voraz y para el Monte Carlo se calcula además el *gap* porcentual respecto al DP, definido como $(u_{DP} - u_{método}) / u_{DP} \times 100$: un gap negativo significa que el método obtuvo **más** utilidad que el DP (lo cual, como se discute más abajo, es un resultado que merece explicación).

12.5 Experimento 4: análisis de robustez

Pregunta: ¿qué tan confiable es una ruta seleccionada si las duraciones reales de los cursos no coinciden exactamente con las duraciones nominales del dataset?

Diseño: para cada instancia se generan varias rutas candidatas con distintas estrategias: la ruta óptima del DP, la ruta del algoritmo voraz, la mejor ruta encontrada por Monte Carlo (con $n_iter = 2000$, semilla 42), y tres rutas “parciales” que se obtienen recorriendo la ruta óptima del DP en orden topológico y quedándose con los primeros cursos hasta llenar el 50 %, el 70 % o el 90 % del presupuesto T_{max} (estas rutas parciales representan, por ejemplo, a un estudiante que decide avanzar de forma más conservadora). Para cada una de estas rutas, y para cada valor de $cv \in \{0,10,0,20,0,30\}$, se ejecuta el análisis de robustez descrito anteriormente con $M = 5000$ simulaciones, registrando la probabilidad de factibilidad $P(\text{factible})$, su intervalo de confianza del 95 % (calculado con el método de Wilson), la duración media simulada, su desviación estándar, el percentil 95 de la duración, y el nivel de riesgo cualitativo (bajo, medio o alto) según los umbrales definidos en el código ($\geq 90\%$ bajo, entre 70 % y 90 % medio, $< 70\%$ alto).

Adicionalmente, para la Instancia B y su ruta óptima del DP, se ejecuta un análisis de sensibilidad más fino, probando ocho valores de cv entre 0.05 y 0.40 con $M = 3000$ simulaciones cada uno, para observar con más detalle cómo decae la probabilidad de factibilidad a medida que aumenta la incertidumbre.

12.6 Procedimiento general y reproducibilidad

En todos los experimentos se sigue el mismo procedimiento: se cargan las instancias ya evaluadas (sin volver a llamar al LLM), se fija una semilla base y se derivan semillas distintas para cada réplica de forma determinista (por ejemplo, `seed = 42 + rep * 73`), se ejecutan las réplicas correspondientes, y se calculan estadísticos agregados (media, desviación estándar, mínimo, máximo). Los resultados de cada experimento se guardan en `results/experimento_N_nombre.json` y `.csv`, y al final se genera un `informe_experimentos.txt` con un resumen de los cinco experimentos y el tiempo total de ejecución (en la corrida documentada en este informe, el tiempo total de los cinco experimentos fue de 16.71 segundos).

13 Resultados y análisis

13.1 Resultados del Experimento 1: iteraciones Monte Carlo

La siguiente tabla muestra los resultados obtenidos para la Instancia B con temperatura fija $T = 1,0$:

n_iter	Utilidad media	Desv. estándar	Tiempo medio (s)	Conv. media (iter)
50	48.90	0.316	0.0025	15.3
100	49.00	0.000	0.0047	19.7
250	49.00	0.000	0.0103	19.7
500	49.00	0.000	0.0219	19.7
1000	49.00	0.000	0.0424	19.7
2000	49.00	0.000	0.0983	19.7
5000	49.00	0.000	0.2171	19.7

El resultado más claro de este experimento es que, con solo 50 iteraciones, la utilidad media ya es 48.90, muy cerca del valor final de 49.00, pero con una pequeña desviación estándar (0.316), es decir, que algunas de las 10 réplicas con 50 iteraciones no llegaron al óptimo. A partir de 100

iteraciones, la desviación estándar se vuelve exactamente 0 en las 10 réplicas: todas encuentran la misma utilidad de 49.00. Además, la columna de convergencia media muestra que, en promedio, la mejor solución ya se encuentra alrededor de la iteración 19.7, es decir, dentro de las primeras 20 simulaciones de cada ejecución, sin importar si el total de iteraciones programadas es 100 o 5000.

Por otro lado, el tiempo de ejecución sí crece de forma aproximadamente lineal con el número de iteraciones: de 0.0025 s con 50 iteraciones a 0.2171 s con 5000, es decir, un factor de aproximadamente 86 veces más tiempo para 100 veces más iteraciones, lo cual es coherente con la complejidad esperada $O(N \cdot |V|^2)$ del muestreo. La conclusión práctica de este experimento es que, para una instancia del tamaño de B, usar más de 250 iteraciones no aporta ninguna mejora en la calidad de la solución, solo más tiempo de cómputo gastado innecesariamente. Esto es información valiosa porque, en `run_fase3.py`, se usan 2000 iteraciones para A y B, y 5000 para C; según este experimento, esos valores son más que suficientes (incluso generosos) para garantizar estabilidad.

13.2 Resultados del Experimento 1b: curvas de convergencia

Las curvas de convergencia confirman y refuerzan la conclusión anterior, pero además permiten ver el comportamiento en las tres instancias:

Instancia	n_cursos	Utilidad final media	Checkpoint de convergencia	Utilidad en checkpoint
A	10	22.0	10	
B	22	49.0	10	
C	35	81.0	10	

En la Instancia A, la curva es completamente plana: desde el checkpoint 10 (es decir, desde la décima simulación) ya se obtiene la utilidad final de 22.0, y se mantiene exactamente igual hasta la iteración 5000. Esto es esperable porque la Instancia A es tan pequeña (10 cursos, $T_{max} = 80$ h) que el espacio de rutas factibles es muy reducido y el muestreo lo cubre casi de inmediato.

En la Instancia B, en el checkpoint 10 la utilidad media de las 3 réplicas es 48.667 (es decir, algunas réplicas todavía no habían encontrado la solución de utilidad 49 en sus primeras 10 iteraciones), pero ya en el checkpoint 25 la media sube a 49.0 y se mantiene así hasta el final. Esto es consistente con la convergencia media de 19.7 observada en el Experimento 1.

En la Instancia C, sorprendentemente, la curva también es plana desde el checkpoint 10, con utilidad 81.0 en todos los puntos de control hasta el 5000. Esto sugiere que, a pesar de ser la instancia más grande (35 cursos, 53 aristas), el muestreo guiado por las utilidades del LLM encuentra muy rápido una región del espacio de búsqueda de alta calidad, y que las iteraciones adicionales (hasta 5000) no logran encontrar nada mejor. En conjunto, el Experimento 1b refuerza la idea de que, para estas tres instancias, el algoritmo Monte Carlo converge extremadamente rápido (en menos de 25 iteraciones en los tres casos), lo cual habla bien de la calidad de la señal de utilidad que aporta el LLM como guía del muestreo.

13.3 Resultados del Experimento 2: temperatura

La siguiente tabla resume los resultados para la Instancia B con $n_{iter} = 1000$ y 15 réplicas por temperatura:

Temperatura	Utilidad media	Desv. estándar	Jaccard medio	Diversidad	Cardinalidad m
0.1	46.0	0.000	1.0000	0.0000	
0.3	48.6	0.507	0.6814	0.3186	
0.5	49.0	0.000	0.8942	0.1058	
1.0	49.0	0.000	0.8942	0.1058	
1.5	49.0	0.000	0.8857	0.1143	
2.0	49.0	0.000	0.8815	0.1185	
3.0	49.0	0.000	0.8942	0.1058	
5.0	49.0	0.000	0.8815	0.1185	

El caso más interesante de esta tabla es $T = 0,1$. Con esta temperatura el muestreo se vuelve casi determinista (siempre elige el curso de mayor utilidad disponible entre los candidatos), lo que produce un índice de Jaccard medio de 1.0, es decir, **las 15 réplicas devuelven exactamente la misma ruta** (diversidad = 0). Pero esa ruta única tiene utilidad 46.0, que es **menor** que el 49.0 que se obtiene con temperaturas mayores. Esto demuestra de forma muy clara el riesgo de un comportamiento puramente voraz: el algoritmo queda atrapado siempre en el mismo óptimo local, que no es el mejor posible.

Con $T = 0,3$ aparece el caso más variable de toda la tabla: la utilidad media es 48.6 con una desviación estándar de 0.507 (la única distinta de cero en todo el experimento), lo que indica que algunas réplicas encuentran la ruta de utilidad 49 y otras se quedan en una de utilidad inferior (probablemente 48 ó 47), y el Jaccard medio cae a 0.6814 (diversidad 0.3186), la mayor diversidad observada. Es decir, $T = 0,3$ es una zona de transición: ya hay suficiente aleatoriedad para escapar del óptimo local de $T = 0,1$, pero todavía no la suficiente para hacerlo de forma consistente en todas las réplicas.

A partir de $T = 0,5$ y hasta $T = 5,0$, la utilidad media se estabiliza en 49.0 con desviación estándar 0 (todas las réplicas encuentran la solución de utilidad máxima observada en este experimento), y el Jaccard medio se mantiene siempre en un rango estrecho entre 0.88 y 0.89 (diversidad entre 0.105 y 0.119). Esto significa que, en este rango de temperaturas, casi todas las réplicas encuentran rutas de la misma utilidad, pero no son exactamente la misma ruta: existe más de una combinación de 8 cursos que alcanza la utilidad 49, y el muestreo con temperatura moderada o alta logra encontrar varias de ellas. La cardinalidad media (número de cursos en la ruta) pasa de 7 cursos con $T = 0,1$ a 8 cursos con $T \geq 0,3$, lo que es consistente con que la ruta de utilidad 46 (con 7 cursos) es estrictamente peor que las rutas de utilidad 49 (con 8 cursos).

La conclusión práctica es que temperaturas muy bajas son peligrosas porque generan un comportamiento determinista que puede quedar atrapado en un óptimo local de baja calidad, mientras que cualquier temperatura mayor o igual a 0.5 da resultados de alta calidad y, además, aporta cierta diversidad de rutas equivalentes en utilidad, lo cual podría ser útil si en el futuro se quisiera ofrecer al usuario más de una alternativa de ruta con la misma utilidad total.

13.4 Resultados del Experimento 3: escalabilidad

La siguiente tabla compara los tres solvers en las tres instancias:

Inst.	n cursos	$n \times W$	DP u	DP t (s)	Voraz u	Voraz gap %	MC u (gap %)
A	10	800	22.0	0.00024	22.0	0.0 %	22.0 (0.0 %)
B	22	4400	34.0	0.00141	48.0	-41.2 %	49.0 (-44.1 %)
C	35	10500	58.0	0.00232	72.0	-24.1 %	81.0 (-39.7 %)

(El tiempo del Monte Carlo, con $n_{iter} = 2000$, fue de 0.0245 s en A, 0.0989 s en B y 0.1569

s en C; es decir, crece con el tamaño de la instancia pero se mantiene siempre por debajo de los 0.2 segundos.)

En la Instancia A, los tres métodos obtienen exactamente la misma utilidad (22.0) y el mismo conjunto de cursos, lo que confirma que para problemas muy pequeños tanto el algoritmo voraz como el Monte Carlo son capaces de encontrar el óptimo verdadero sin dificultad.

En las instancias B y C aparece un resultado que, a primera vista, parece contradictorio: tanto el algoritmo voraz como el Monte Carlo obtienen **una utilidad mayor** que el DP exacto (48.0 y 49.0 frente a 34.0 en B; 72.0 y 81.0 frente a 58.0 en C), lo que produce los valores de “gap” negativos de la tabla. En teoría, un algoritmo exacto de programación dinámica debería encontrar siempre una solución igual o mejor que cualquier heurística, nunca peor, así que un gap negativo no debería ser posible si todos los métodos están resolviendo exactamente el mismo problema con la misma función objetivo.

La explicación más probable, que ya se había anticipado como una posible inconsistencia al describir la Fase 3, es que la implementación del DP exacto utilizada en este experimento para $n > 10$ corresponde a la **versión heurística** de `baseline.py` (la que usa una tabla (i, w) y luego aplica una corrección de clausura de prerequisites después del backtracking), y no a la versión exacta por máscara de bits, que solo se activa para $n \leq 20$. La Instancia B tiene 22 cursos y la Instancia C tiene 35, ambas por encima de ese umbral, por lo que en ambos casos el “DP” que se está comparando es en realidad la versión heurística, que **no garantiza optimalidad** y que, además, puede perder cursos durante el paso de corrección de prerequisites, lo cual explicaría por qué su utilidad queda muy por debajo de las otras dos heurísticas. En la Instancia A (10 cursos), en cambio, sí se usa la versión exacta por máscara de bits, y por eso ahí los tres métodos coinciden.

Más allá de esta inconsistencia (que se discute también en la sección de limitaciones), el experimento sí deja una conclusión útil: tanto el algoritmo voraz como el Monte Carlo escalan muy bien en tiempo (siempre por debajo de los 0.16 segundos incluso en la instancia de 35 cursos), y el Monte Carlo es consistentemente el método que obtiene la mayor utilidad de los tres en las instancias B y C, lo que justifica su uso como solver principal para la Instancia C en `run_fase3.py`.

13.5 Resultados del Experimento 4: robustez

Para la Instancia A, la ruta óptima (que coincide con la ruta del DP, del voraz y del Monte Carlo, ya que en esta instancia los tres métodos llegan al mismo resultado) muestra la siguiente sensibilidad a la incertidumbre en las duraciones:

cv	P(factible)	Nivel de riesgo	Observación
0.10	87.36 %	MEDIO	
0.20	74.14 %	MEDIO	
0.30	67.40 %	ALTO	

Esto significa que la ruta óptima de la Instancia A usa casi todo el presupuesto de 80 horas (poca holgura), y por eso su factibilidad se degrada con bastante rapidez cuando aumenta la variabilidad de las duraciones: ya con un 30 % de variación típica, hay más de un 30 % de probabilidad de que la ruta termine excediendo las 80 horas disponibles.

En contraste, las rutas **parciales** (50 %, 70 % y 90 % del presupuesto, derivadas de la ruta óptima) para la Instancia A tienen $P(\text{factible}) \geq 99,9\%$ en los tres niveles de cv probados, lo cual es esperable: al usar menos del presupuesto total, queda mucha holgura para absorber variaciones en las duraciones.

Para la Instancia B, el patrón cambia de forma interesante: la ruta óptima del DP (que en este caso, recordemos, corresponde a la versión heurística con utilidad 34.0) tiene $P(\text{factible}) \geq 99,98\%$ en los tres valores de cv , es decir, es extremadamente robusta, porque usa muy poco

del presupuesto de 200 horas disponibles (mucho holgura). Sin embargo, tanto la ruta voraz como la mejor ruta de Monte Carlo (que tienen utilidad mucho mayor: 48.0 y 49.0) muestran $P(\text{factible})$ alrededor de 51 %, clasificado como riesgo ALTO en los tres niveles de cv . Esto revela un **trade-off claro entre utilidad y robustez**: las rutas que aprovechan mejor el presupuesto de tiempo (y por tanto obtienen más utilidad) tienen, naturalmente, menos margen para absorber variaciones, mientras que una ruta que deja mucho presupuesto sin usar es muy robusta pero deja mucho valor sobre la mesa.

Para la Instancia C se observa un patrón mixto: la ruta voraz tiene $P(\text{factible})$ que va de 95.82 % ($cv = 0,10$, riesgo bajo) a 81.26 % ($cv = 0,20$, riesgo medio) y 72.80 % ($cv = 0,30$, riesgo medio), mientras que la mejor ruta de Monte Carlo, que tiene la mayor utilidad (81.0) de todo el experimento de escalabilidad, muestra $P(\text{factible})$ alrededor de 51-52 % (riesgo alto) en los tres niveles de cv . De nuevo se confirma el mismo trade-off: la ruta de mayor utilidad es también la más ajustada al presupuesto y, por tanto, la más sensible a la incertidumbre.

El análisis de sensibilidad adicional para la Instancia B (ruta del DP, con cv entre 0.05 y 0.40) muestra que $P(\text{factible})$ se mantiene en 1.0 (100 %) para $cv \leq 0,25$, baja levemente a 0.9997 con $cv = 0,30$ y $cv = 0,35$, y a 0.9973 con $cv = 0,40$. Esto confirma que esta ruta en particular tiene tanta holgura que incluso una variabilidad muy alta en las duraciones (± 40 %) apenas afecta su factibilidad.

En conjunto, el Experimento 4 deja una conclusión importante para el diseño del sistema: la utilidad total no debería ser el único criterio para recomendar una ruta al usuario. Sería razonable que el sistema, además de mostrar la ruta de mayor utilidad, también muestre su nivel de riesgo (por ejemplo, “esta ruta tiene una utilidad de 49 pero solo un 51 % de probabilidad de completarse dentro de las 200 horas si las duraciones varían un 20 %”), para que el usuario pueda decidir si prefiere esa ruta o una alternativa más conservadora con menor utilidad pero mayor probabilidad de éxito.

14 Limitaciones y posibles mejoras

Como en cualquier proyecto, existen limitaciones que vale la pena mencionar honestamente.

La primera, y la más relevante a la luz de los experimentos, es la **inconsistencia entre la programación dinámica y las heurísticas observada en el Experimento 3**. Como se explicó en el análisis de resultados, todo indica que para las instancias B y C (más de 20 cursos) la comparación se hizo contra la versión *heurística* de la programación dinámica (que no garantiza optimalidad y puede romper la clausura de prerrequisitos al reconstruir la solución) en lugar de contra la versión exacta por máscara de bits. Esto hace que los valores de “gap” reportados no representen realmente una distancia al óptimo verdadero, sino una comparación entre dos heurísticas distintas. Una mejora directa sería forzar la versión exacta por máscara de bits también para $n = 22$ y $n = 35$ (su costo, 2^{22} y 2^{35} , sigue siendo computacionalmente intratable para $n = 35$, por lo que en ese caso sería necesario un solver exacto distinto, como uno basado en programación lineal entera).

La segunda limitación es que, al ser el problema NP-difícil, la implementación exacta por máscara de bits solo es práctica para instancias de hasta unos 20 cursos; para instancias más grandes se depende de heurísticas (voraz y Monte Carlo) que, aunque en los experimentos obtuvieron buenos resultados, no garantizan encontrar la mejor solución posible, y no existe en este proyecto una cota superior teórica independiente con la que contrastarlas.

La tercera limitación es que el componente LLM depende de credenciales externas y de la disponibilidad de proveedores externos (OpenAI, Gemini, Groq), y su calidad puede variar según el modelo usado; los experimentos de optimización, sin embargo, se ejecutaron sobre instancias ya evaluadas, por lo que no se ven afectados directamente por la disponibilidad del LLM en el momento de experimentar.

La cuarta limitación, señalada en el Experimento 4, es que el análisis de robustez asume

que las duraciones de los distintos cursos varían de forma **independiente** entre sí (cada curso se modela con su propia distribución Gamma sin correlación con los demás), lo cual es una simplificación: en la práctica, si una persona tiende a tardar más de lo esperado en un curso, es razonable pensar que también tardará más en los siguientes (por ejemplo, por falta de tiempo disponible en su semana), lo que generaría una correlación positiva entre las duraciones reales que el modelo actual no captura.

Finalmente, al ser un dataset sintético, es posible que no capture todos los patrones de dependencias que existirían en un catálogo de cursos real, y los valores concretos de utilidad obtenidos dependen del objetivo de usuario usado durante la evaluación LLM (en este caso, un perfil orientado a NLP y LLMs), por lo que los resultados numéricos de los experimentos (por ejemplo, “utilidad 49 para la Instancia B”) son específicos de ese objetivo y cambiarían si se usara un perfil distinto.

Como posibles mejoras a futuro, derivadas directamente de los experimentos realizados, se proponen las siguientes: (1) incorporar un solver de programación lineal entera (ILP) o de programación por restricciones (CP) para tener una referencia óptima rigurosa también en instancias de 22 y 35 cursos, y así corregir la inconsistencia del Experimento 3; (2) usar los resultados del Experimento 1 para fijar un número de iteraciones más eficiente (por ejemplo, 250 en lugar de 2000-5000) sin pérdida de calidad, reduciendo el tiempo de cómputo sin afectar la utilidad; (3) usar los resultados del Experimento 2 para evitar temperaturas por debajo de 0.5 en producción, ya que se demostró que pueden producir soluciones de menor calidad de forma consistente; (4) incorporar el nivel de riesgo del Experimento 4 como parte de la salida que se le muestra al usuario, posiblemente ofreciendo más de una ruta (la de mayor utilidad y una alternativa más robusta) para que el usuario elija; (5) modelar correlación entre las duraciones de los cursos en el análisis de robustez; y (6) repetir el conjunto completo de experimentos con distintos objetivos de usuario, para comprobar si las conclusiones (por ejemplo, la convergencia rápida del Monte Carlo, o el trade-off utilidad-robustez) se mantienen para perfiles distintos al de NLP/LLMs usado aquí.

15 Conclusión

Este proyecto demuestra que es posible combinar un modelo de lenguaje grande con un algoritmo clásico de optimización para resolver un problema que ninguno de los dos podría resolver bien por separado. El modelo de lenguaje aporta la comprensión semántica de lo que el usuario realmente quiere aprender, traduciendo eso en un número de utilidad para cada curso, mientras que el algoritmo clásico se encarga de la parte matemática: encontrar la mejor combinación de cursos posible respetando el tiempo disponible y las dependencias entre ellos. La separación clara de responsabilidades entre ambos componentes hace que el sistema sea más fácil de entender, de depurar y de mejorar en el futuro.

El conjunto de cinco experimentos realizados permitió ir más allá de una sola ejecución de prueba y caracterizar el comportamiento del sistema de forma sistemática: se determinó cuántas iteraciones de Monte Carlo son realmente necesarias (Experimento 1 y 1b), se identificó el rango de temperaturas que da resultados estables y de buena calidad (Experimento 2), se comparó la escalabilidad de los tres solvers en instancias de tamaño creciente (Experimento 3), y se cuantificó el riesgo de las distintas rutas frente a la incertidumbre en las duraciones, descubriendo un trade-off claro entre utilidad y robustez (Experimento 4). Al mismo tiempo, los experimentos sacaron a la luz una inconsistencia real en la implementación (la comparación contra la versión heurística de la programación dinámica en lugar de la exacta para instancias grandes), lo cual es en sí mismo un resultado valioso: permite documentar con precisión qué parte del sistema necesita revisión antes de considerarlo listo para un uso más amplio, y deja un camino claro de mejoras concretas para el trabajo futuro.